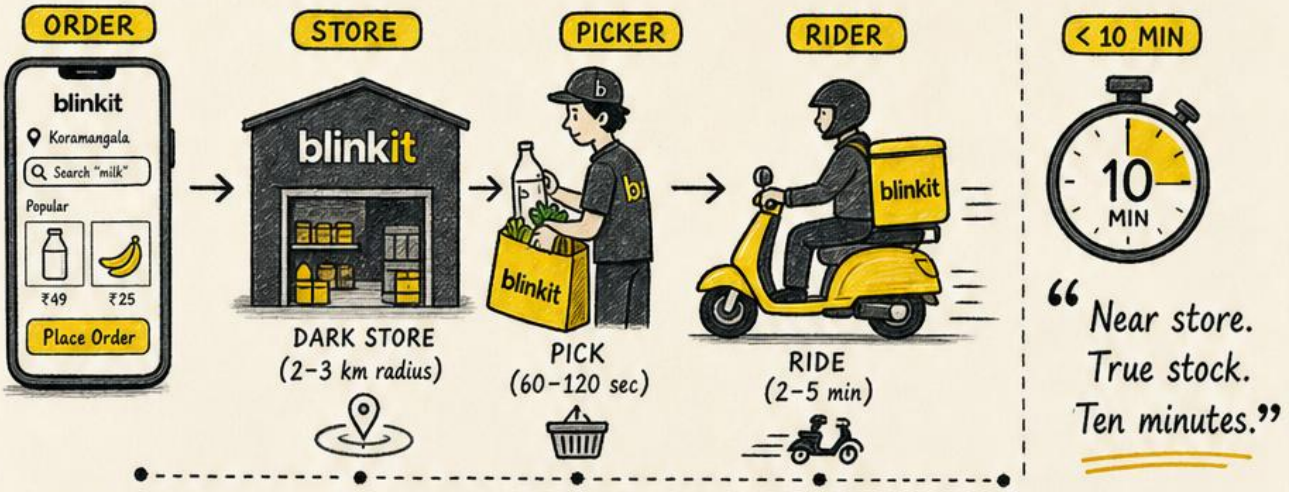
@ codedsoul_05

QUICK COMMERCE SD v1.0 ☆

BLINKIT SYSTEM DESIGN

— HANDWRITTEN HANDBOOK —

Designing a 10-Minute Hyperlocal Grocery Delivery Platform



Dark Stores • Inventory • Picking • Dispatch • ETA • Promise

What's Inside?

1. Requirements & Goals
2. High Level Architecture
3. Data Model & Catalog
4. Inventory & Stock Truth
5. Dark Store Network
6. Order & Cart Flow
7. Picking & Packing
8. Dispatch & Routing
9. ETA & Promise Engine
10. Observability & SLA

+ Scaling, Reliability & Trade-offs ☆

Blinkit by the Numbers

- Promise: < 10 minutes (doorstep delivery)
- Operating radius: 2–3 km (hyperlocal)
- Dark stores within 2–3 km radius
- Demand is spiky (peaks 8–12 PM)

Small radius. Tight SLA. High frequency.

Topics Covered

- ☑ Geo Search (Near Store)
- ☑ True Stock & Availability
- ☑ Menu Catalog & Pricing
- ☑ Order State Machine
- ☑ Picker Assignment
- ☑ Batching & Wave Picking
- ☑ Dispatch & Routing
- ☑ Live Location Tracking
- ☑ ETA Prediction
- ☑ Surge, Promos & Fees
- ☑ Payments & Refunds
- ☑ Cancellation & Failures
- ☑ Reliability & Blitz Scale

Who is this for?

- ☆ System Design Interview Preparation
- ☆ Backend / Full Stack Developers
- ☆ Anyone building at scale

"Zomato waits on a kitchen. Blinkit waits on a picker in a room you control."

Select Near. Pick Fast. Deliver Now. ☆

01 — REQUIREMENTS & SCOPE

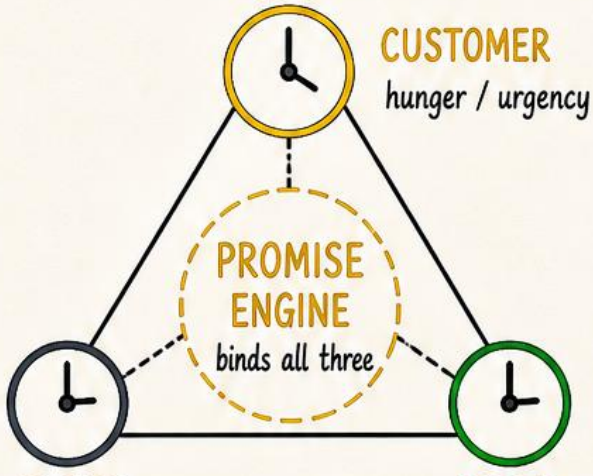
Show only what can leave a nearby dark store and reach the customer in about 10 minutes.

CORE FEATURES

- Hyperlocal catalog by location
- Real-time availability & search
- Cart with live promise ETA
- Checkout & payment
- Picker assignment & packing
- Rider assignment & tracking
- Substitutions / missing items
- Order rating & support

blinkit

THREE CLOCKS



Unlike food delivery, preparation is picking in YOUR store — more controllable, still bursty.

FUNCTIONAL REQUIREMENTS

1. Discovery

store + SKU availability

2. Commerce

cart, price, checkout

3. Fulfilment

pick, pack, handoff

4. Delivery

assign, ride, complete

NON-FUNCTIONAL REQUIREMENTS

- Promise latency — ETA in ~100s of ms at browse
- Spiky demand — rain, IPL, evenings
- Inventory accuracy — oversell kills trust
- Strong consistency on reserve/pick
- Read-heavy browse; write-heavy at order
- Payments + address privacy

OUT OF SCOPE

- ✗ Long-haul warehouse retail (Amazon-style)
- ✗ Restaurant marketplace
- ✗ National slow parcel courier
- ✗ Dark store real-estate strategy details

INTERVIEW TIP

Open with the contrast: Blinkit is not Zomato with groceries. It is Amazon inventory semantics inside a 2 km radius with a stopwatch.

Hyperlocal truth beats infinite catalog. ★

02 — CAPACITY ESTIMATION

Traffic is city-local and spiky — a single dark store can saturate even when the country average looks fine.

ASSUMPTIONS

Orders / day (platform)	Peak share in 4 hours	Avg items / order	Browse : order	Dark stores	Active riders at peak
= 2 Million (illustrative)	= 50%	= 8	= 30 : 1	= 1,000	= 100,000

Illustrative interview numbers.

ORDER TPS

Average = 2M ÷ 86,400
≈ 23 orders/sec

Peak (50% in 4h): 1M ÷ 14,400
≈ 70 orders/sec

City/store bursts much higher locally

Provision per store throughput (picks/hour), not only global TPS.

BROWSE / SEARCH

2M × 30 = 60M browse events/day

≈ 700/sec avg; several thousand/sec peak

Cache store-local catalogs aggressively

INVENTORY WRITES

Each order reserves ≈ 8 SKUs → peak ≈ 560 reserve ops/sec global

Plus picker scans, restocks, waste adjustments

Hot SKUs (milk, bread, eggs) concentrate locks

Hot SKU rows need careful concurrency design.

PICK OPERATIONS

2M orders × 8 scans

≈ 16M scan events/day

At peak, picker apps generate heavy websocket/API traffic per store

LOCATION PINGS

100k riders × ping/5s
≈ 20k writes/sec at peak

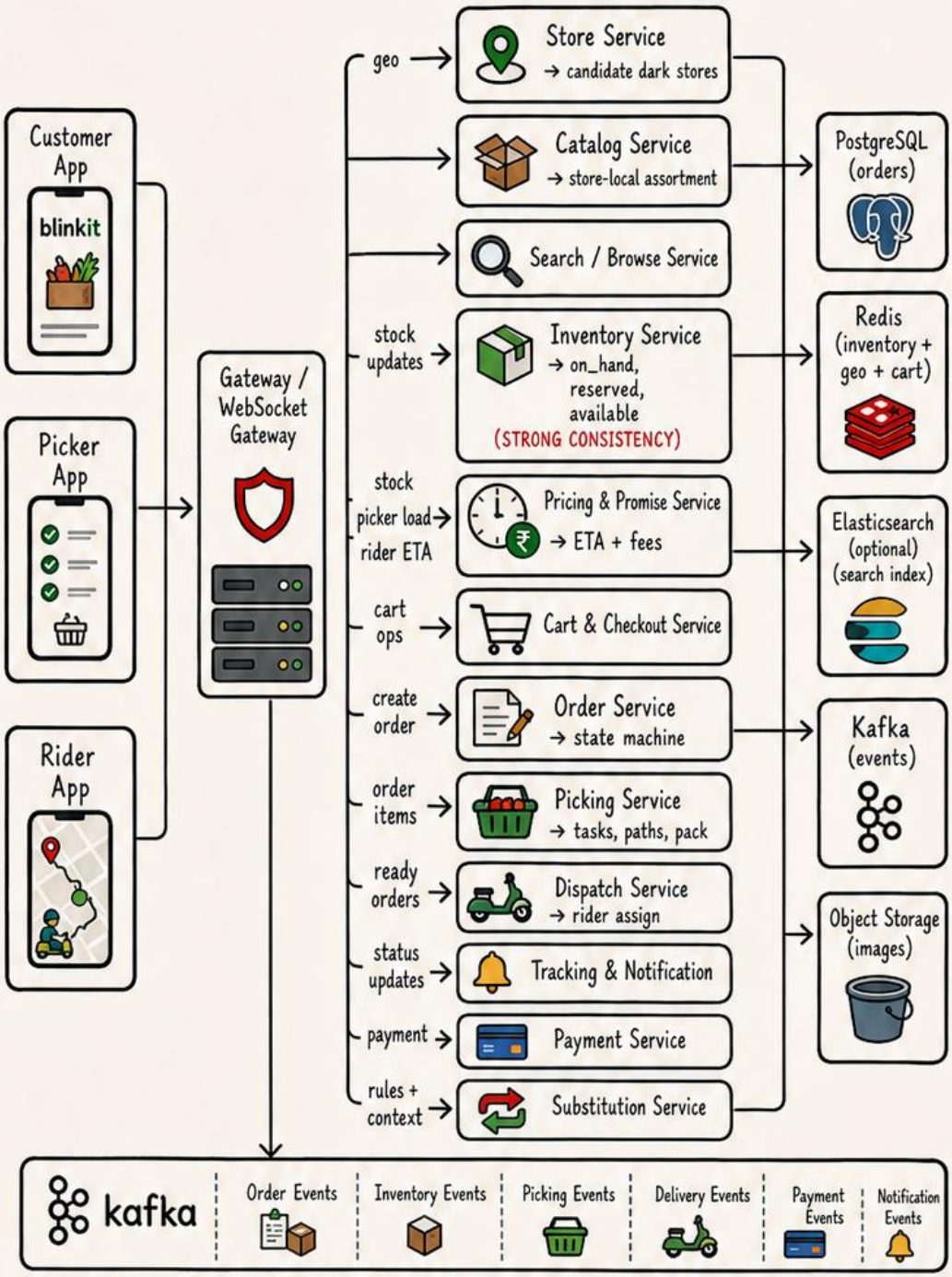
Keep last-location hot in Redis geo

WHAT THESE NUMBERS TELL US

Store-level capacity is the real bottleneck	Catalog reads → cache by store_id	Inventory → short transactions, avoid fat locks	Rider geo → in-memory	Promise engine must be cheap on every cart mutation

Scale cities and stores — not one giant warehouse. ★

03 — HIGH-LEVEL ARCHITECTURE



WHY THIS SHAPE?

- Store selection precedes catalog — assortment is local.
- Inventory isolated — oversell prevention.
- Picking is a first-class system, not a note on the order.
- Promise service combines stock, picker load, rider ETA.
- Kafka for analytics/ notifications; checkout path stays sync and short.

EXTERNAL SERVICES

- Maps / ETA (Geo & Traffic)
- Payment Gateway
- Push / SMS (FCM / APNS)

ORDER FLOW (HAPPY PATH)

1. Locate store
2. Browse
3. Cart + promise
4. Pay
5. Reserve stock
6. Pick
7. Pack
8. Assign rider
9. Deliver
10. Complete

KEY DESIGN

- Strong consistency for inventory & reservations.
- Short, synchronous path till payment & reservation.
- Everything after reservation is event-driven.
- Design for high QPS and low latency at store level.

INTERVIEW TIP

Draw picker app explicitly. Candidates who only draw customer + rider miss quick commerce.

Local store, strict stock, parallel pick and ride. ★

04 — APIs & CONTRACTS

Every browse call is store-scoped. Every order call is idempotent.

CUSTOMER APIs

- GET /v1/stores/nearby?lat=&lng=
- GET /v1/stores/{store_id}/catalog?cursor=
- GET /v1/search?q=&store_id=
- POST /v1/cart/items
- GET /v1/cart/promise — ETA, fees, unavailable items
- POST /v1/orders — Idempotency-Key
- GET /v1/orders/{id}
- POST /v1/orders/{id}/substitutions/preference

PICKER APIs

- GET /v1/picker/tasks
- POST /v1/picker/tasks/{id}/scan — {sku, qty}
- POST /v1/picker/tasks/{id}/missing
- POST /v1/picker/tasks/{id}/pack_complete

RIDER APIs

- POST /v1/rider/status — online/offline
- POST /v1/rider/orders/{id}/accept
- POST /v1/rider/orders/{id}/picked_from_store
- POST /v1/rider/orders/{id}/delivered
- POST /v1/rider/location — high frequency

ORDER CREATE (JSON)

```
{
  "idempotency_key": "ord_9f2a",
  "store_id": "ds_118",
  "cart_id": "crt_55",
  "address_id": "adr_3",
  "payment_method": "upi"
}
```

STORE HEADER



required on catalog/ search

PROMISE RECHECK



at checkout, not only at add

IDEMPOTENCY



pay/ order

PARTIAL AVAILABILITY



return line-level errors

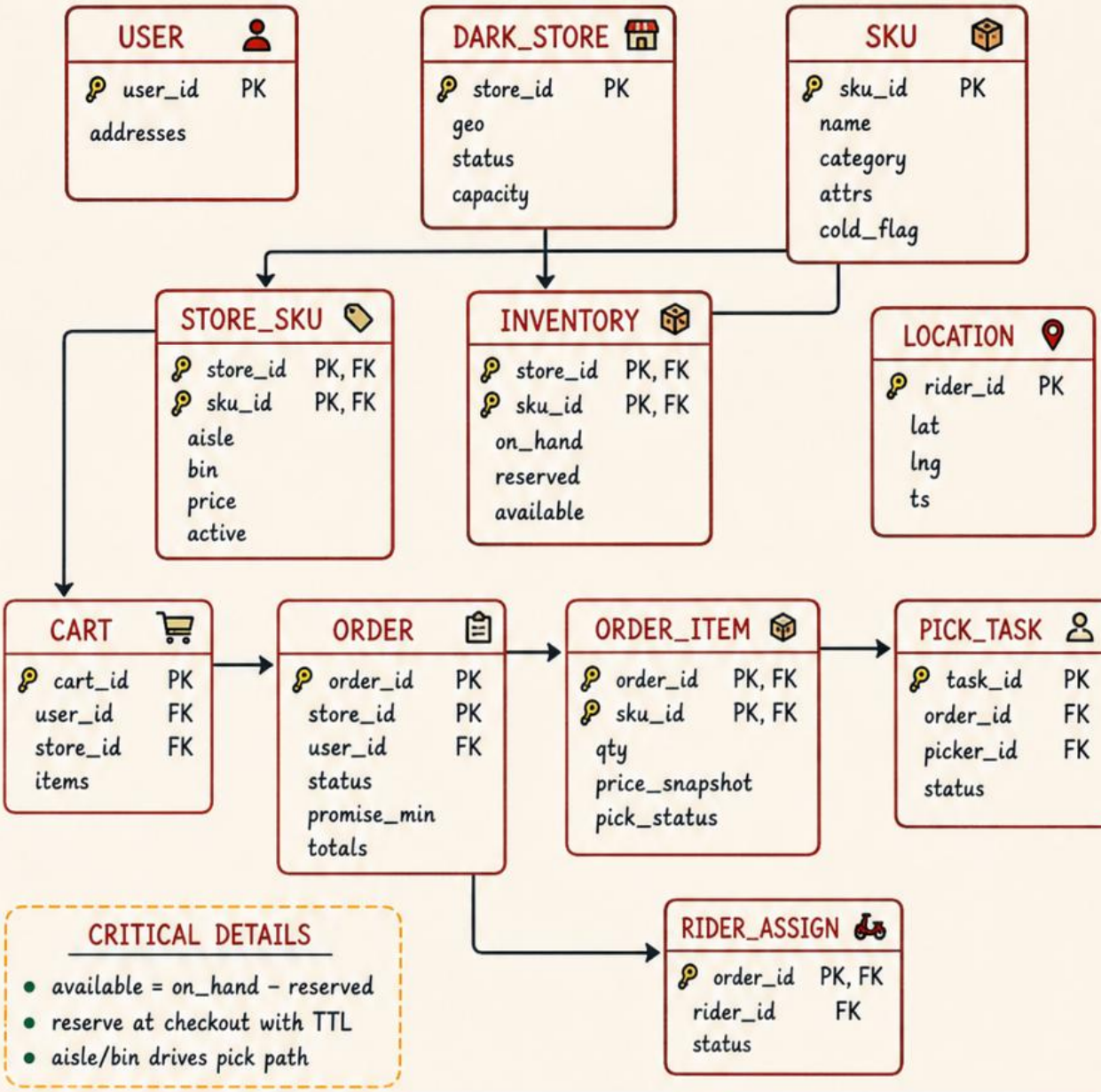


Never trust cart availability from 5 minutes ago — milk sells out continuously.

Store-scoped reads. Idempotent writes. Fresh promise. ★

05 — DATA MODEL & STORAGE

Assortment is per dark store. Inventory is per SKU per store. Orders are a ledger.



CRITICAL DETAILS

- available = on_hand − reserved
- reserve at checkout with TTL
- aisle/bin drives pick path

STORAGE CHOICES

- **PostgreSQL** – orders, payments, users, stores (ACID)
- **Redis** – inventory counters, geo index, cart, sessions
- **Kafka** – events stream: orders, inventory, tasks
- **Elasticsearch** (opt.) – search, autocomplete, catalog
- **CDN/Object Storage** – product images, docs

PARTITIONING

- Partition by city_id
- Then by store_id
- Keep store-scoped data local
- TTL for reservations in Redis
- Archive orders after 6–12 months

ACCESS PATTERNS (DRIVE THE DESIGN)

1. Nearby store discovery
2. Store catalog page
3. Reserve multi-SKU at checkout
4. Picker task list
5. Live order tracking & status

Never global inventory — only store-local truth.

Store-scoped stock. Order-scoped history.

06 — DARK STORE NETWORK

The first product decision is which store can serve you — not which brand you typed.



STORE SELECTION FLOW

Lat/Lng (of customer) → Candidate stores from geo index (within 3 km) → Filter open + serviceable polygons → Score by ETA (picker wait + ride) + load → Pick primary store (sometimes backup)

WHY MICRO-FULFILMENT

- Stores live closer to customers
- Smaller inventory, faster turns
- Low picker travel time
- Enables 10–20 min promises
- Better freshness & local demand

SERVICEABLE POLYGONS, NOT RAW RADIUS

- Respect roads, rivers, flyovers, tolls
- Use drive time, not straight line
- Each store has a dynamic service area

STORE CAPACITY

Pickers × productivity + Rider supply = Orders per hour

- Pickers x productivity (UPH)
- Plus rider supply in the zone
- Load = projected orders / capacity

SOFT CLOSE WHEN OVERLOADED

- If load > threshold, stop new promises
- Hide store from discovery for that zone
- Existing carts may switch to backup
- Protect OTIF and customer trust

ASSORTMENT DIFFERS BY STORE

- Local demand drives catalog
- Size, category mix, planograms vary
- Show only what the selected store can deliver in promise

NOTE (IMPORTANT!)

Showing an item from a store that cannot deliver in promise is a bug.

Geo select, then catalog. Never the reverse.

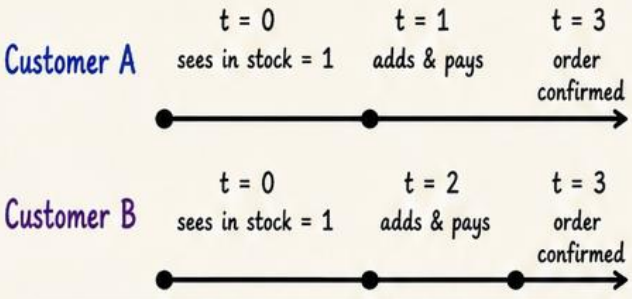
07 — INVENTORY & RESERVATION

A wrong 'in stock' at minute zero becomes a cancelled item at minute three.

THE RACE CONDITION

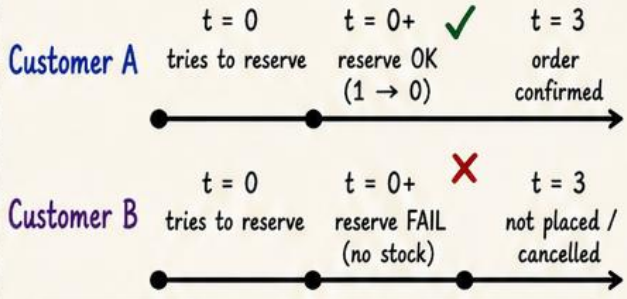
Last unit of Milk in Store S1

WITHOUT ATOMIC RESERVE → DOUBLE SELL



Both orders confirmed.
Stock went negative. One will be cancelled.

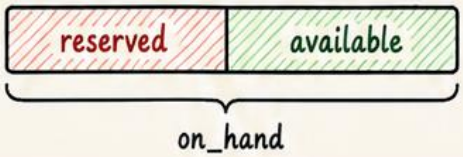
WITH CONDITIONAL UPDATE → ONE WINS



Only one order confirmed.
No negative stock.

THE THREE-STATE MODEL

- on_hand — physically in the warehouse
- reserved — promised (not yet sold)
- available = on_hand − reserved



RESERVATION LIFECYCLE

1. **RESERVE** at checkout
   Place a hold with TTL (5–10 min).
2. **COMMIT** on payment / pick confirm
   Policy: on payment success OR at pick confirm (store policy).
3. **RELEASE** on cancel / timeout / missing
   On user cancel, TTL expiry, or address failure — release the hold.

ATOMIC RESERVE (SQL-STYLE)

```
UPDATE inventory
   SET  reserved = reserved + 1
  WHERE store_id = :store_id
    AND sku_id   = :sku_id
    AND (on_hand - reserved) >= 1;
```

Rows affected = 1 → reserve success
0 → out of stock (no reserve)

HOT SKUs: HANDLE WITH CARE

- Milk, Eggs, Bread, etc. are hot SKUs.
- Consider sharded counters or careful row locking.
- Keep transactions short. No network calls inside txn.
- Avoid long-running locks. Retry on conflict.
- Measure p95 commit latency. Tune indexes.

RESTOCK & WASTE ADJUSTMENTS

Append inventory events for restock, transfers, damage, expiry, returns. All adjustments go through the same model (on_hand / reserved).

CYCLE COUNTS FIX DRIFT

Physical counts drift over time. Run cycle counts and post adjustments to correct on_hand.
Trust events. Reconcile regularly.

DO NOT RESERVE ON ADD-TO-CART

Carts are idle.
Reserving here freezes stock and hurts conversion.
Reserve only at checkout.

STORE-LOCAL ROWS

Keep inventory rows scoped by store (and SKU). Reservation must deduct from the same store row to keep availability correct.

Atomic reserve. Short TTL. Store-local rows.

08 — CATALOG, SEARCH & AVAILABILITY

Browse a small store catalog at massive QPS — filter by what is actually pickable now.

DATA FLOW — FROM CATALOG TO LIVE AVAILABILITY

① Global SKU master (canonical truth)

- SKU, title
- brand, category
- pack, price MRP
- images, attrs
- active flag

→

② Store assortment (STORE_SKU)

- which SKUs sold here
- shelf / aisle mapping
- local price & pack
- active / delisted

→

③ Live availability overlay (inventory / Redis)

- available_quantity
- reserved buffer
- status (ACTIVE/OUT)
- next restock ETA

→

Effective availability

Pickable Now

= available_quantity − reserved_buffer > 0

SEARCH FLOW

① User query (text / voice)

milk

- spellfix
- synonyms
- typo tolerant

→

② Store-scoped search index (per store)

- SKU, title, brand
- category, tags
- attributes
- embeddings (opt)

→

③ Filter available > 0 and active

- available_qty > 0
- status = ACTIVE
- not delisted

→

④ Rank (lightweight)

- popularity (views)
- margin / GM%
- personalization (light)
- freshness / promos

→

⑤ Facets (categories)

- category
- brand
- price
- pack size

→

⑥ Results with availability

0

- show stock badge
- ETA if low stock

CACHING STRATEGY

- Cache category / listing pages per store
- Key: store_id + category_id + filters
- TTL: 10–30 seconds (tune by traffic)
- Do NOT cache "in stock" for long
- Merge live availability at read time

Cache (pages) → Cached page + Live availability (Redis) = Final response

IMAGES & CONTENT

- Product & category images on CDN
- Multiple sizes, WebP
- Lazy load & progressive render

SPELLFIX & NATURAL QUERY

milk

- Handle grocery typos & slang
- "maggie", "maggi", "meggi" → maggie
- "amul mlk", "tond dudh" → milk
- Fuzzy match + synonym dictionary

OUT-OF-STOCK EXPERIENCE

- If item is out of stock
  - Show similar in-store alternatives
  - Same category, same need
  - Consider pack size & price band

CONSISTENCY NOTE

Search index can lag.
Availability must not.

→ Always overlay live availability at read time.

★ Cache the aisle. Freshen the stock flags. ★

09 — CART, PROMISE & CHECKOUT

★ The yellow '10 minutes' badge is a computed promise — recompute as the cart changes.

PROMISE BREAKDOWN

Store selection	Picker queue wait	Pick time (~ items)	Pack	Rider travel (road ETA)	Buffer (variability)	PROMISE
1 min	2 min	3 min	1 min	4 min	1 min	10 min

Example: 1 + 2 + 3 + 1 + 4 + 1 ≈ 12 min ⟶ Show window 10–15 min or still show "10 min" if policy allows with buffer.

CHECKOUT SEQUENCE

1	Add items to cart	User adds/updates items.
2	Validate store	Find best store by availability, distance & policy.
3	Recompute promise & fees	Promise (10–15), delivery fee, taxes, offers.
4	Checkout	User proceeds to checkout page.
5	Re-validate stock & prices	Prices, stock and offers can change.
6	Reserve	Soft-reserve inventory & freeze price (TTL).
7	Pay	Collect payment (UPI/Card/Wallet/COD).
8	Create order	Persist order, payment status = PLACED.
9	Notify store (picking)	Send order to store for picking.

SERVER-SIDE PRICING

- All price, tax, offer & discount calculation happens on the server.

DELIVERY FEE (SURGE)

- Distance, demand, weather, store load & time of day affect delivery fee.

RAIN FEE

- Applied in heavy rain zones to protect rider incentives.

MINIMUM ORDER

- Ensure basket value meets store minimum (order acceptance rule).

IDEMPOTENCY

- Use idempotency_key → prevent double charges & duplicate orders.

PARTIAL RESERVE HANDLING

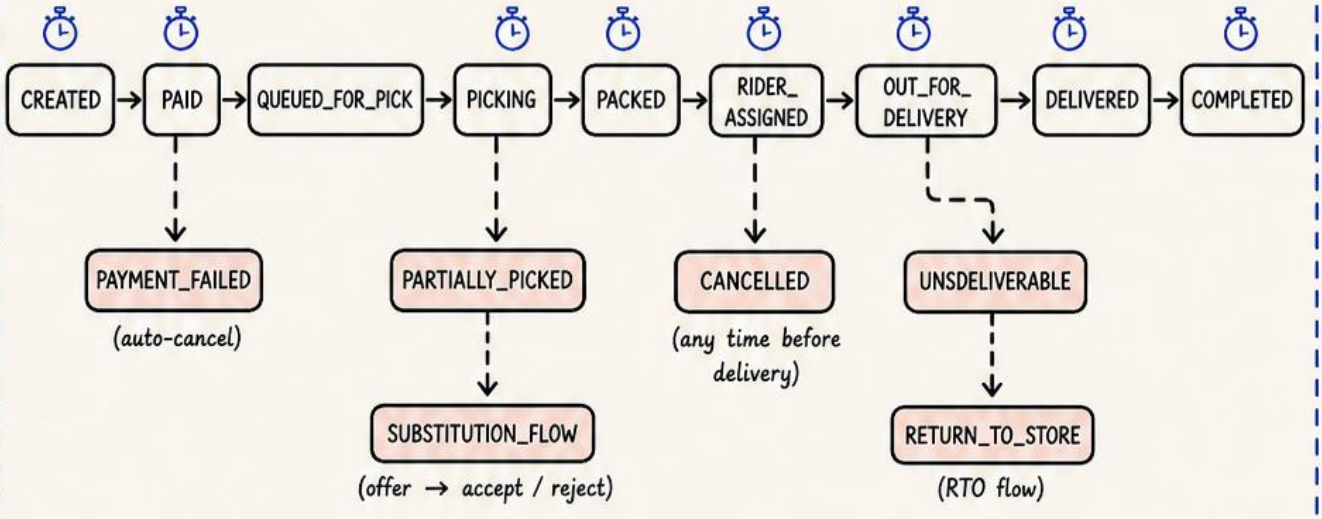
- If some SKUs fail reserve → return partial error and adjust cart & promise.

INTERVIEW TIP Promise engine is the product differentiator — mention picker load, not only road ETA.

Recompute promise, then reserve, then charge. ★

10 — ORDER LIFECYCLE

One order, three apps — customer, picker, rider — one state machine.



Event log (ORDER_EVENT) is append-only.
Every state change = one event. No updates, only new events.

WHO TRIGGERS WHAT

- **Customer** → Cancel (early), address change (early), support actions
- **Picker** → Scan items, mark missing, request substitution
- **Rider** → Accept, pickup, on the way, delivered, undeliverable
- **System** → Timeouts, auto-cancel, reassign, RTO triggers

EVENTS WE RECORD

- Append an ORDER_EVENT for every legal state transition.
- Includes: actor, timestamp, payload, app_source.
- Immutable, ordered by time. This is the source of truth.
- Used to rebuild state and timelines for any order.

TIMEOUTS THAT MATTER

- PICK SLA breach → alert picker leads, re-queue or reassign.
- RIDER NOT FOUND (no accept) → widen search radius, retry, or auto-cancel.
- DELIVERY TIMEOUT → trigger undeliverable → RTO flow.

CANCELLATION RULES

- Customer can cancel before PICKING → full refund.
- After PICKING starts → cancel = not allowed or partial refund (as per policy).
- After RIDER_ASSIGNED → cancel not allowed.
- Cancelled orders stop further transitions. Cancellation event is final.

EDGE CASES

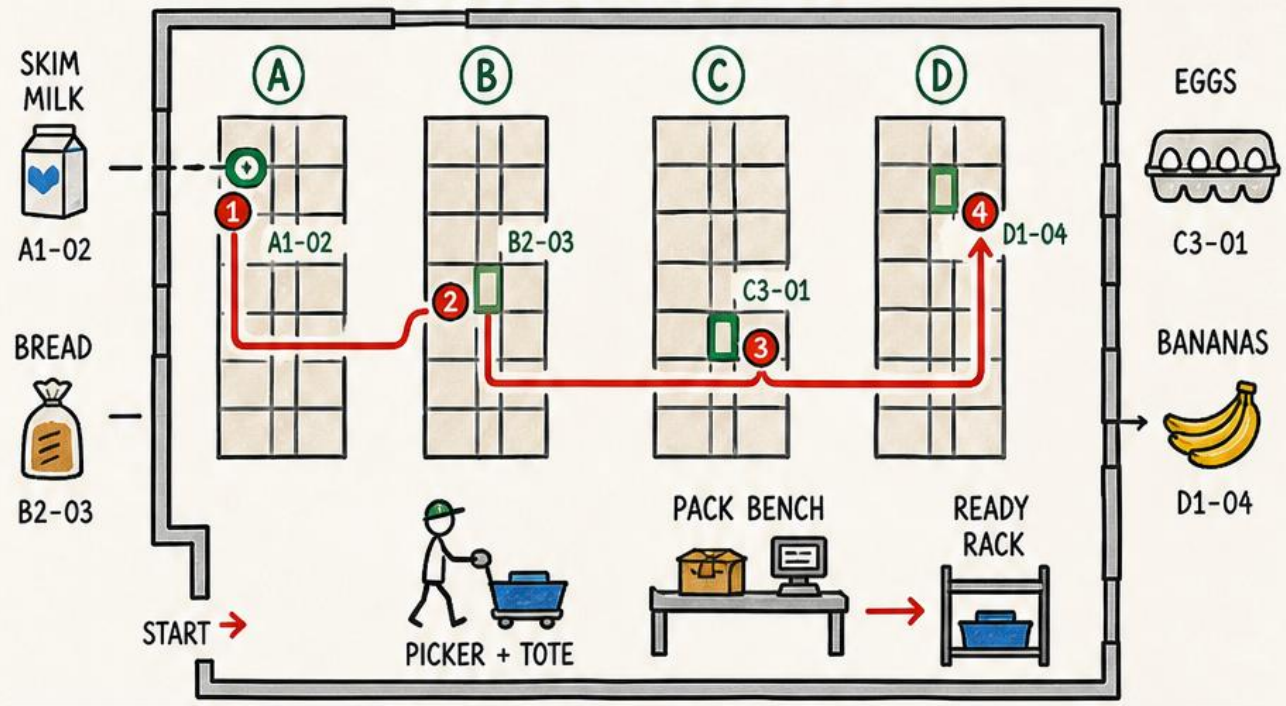
- Payment fails after creation → auto-cancel.
- Partial pick + no acceptable substitution → cancel (if allowed).
- Rider pickup but no movement → reassign or cancel.
- RTO completed → return updated, order marked accordingly.

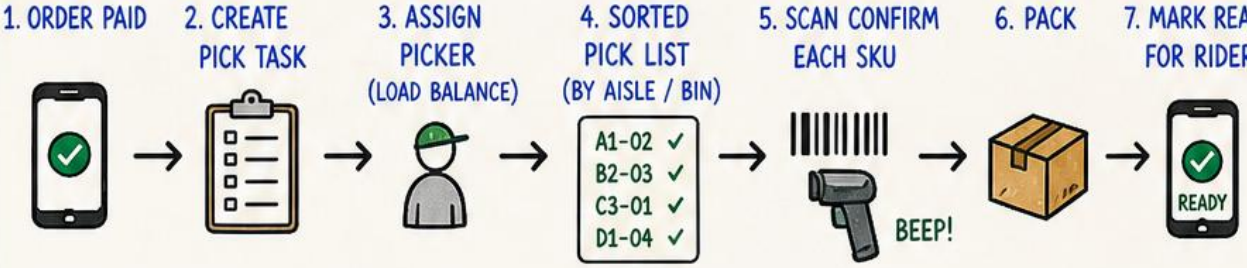
Legal transitions only; everything else is an event.

11 — PICKING & DARK STORE OPS ★ KEY PAGE

The warehouse is small — the path still matters when every minute is the product.

DARK STORE FLOOR (EXAMPLE)





BATCHING PICK LISTS

Across 2 orders ONLY if:

- ✓ Same aisle cluster (<= 6 aisle gap)
- ✓ Promise window allows

Usually single-order picks for 10-min SLA.

Order A + Order B

OK if same cluster

Too far apart

MISSING ITEM?

1) Check substitution rules

2) If no suitable sub:

- Remove item
- Refund to customer

Communicate via app (notification + reason).

COLD ITEMS LAST

Pick refrigerated / frozen items in the end.

Use insulated bags for cold.

PICKER PRODUCTIVITY

- Lines per hour (LPH)
- Accuracy %
- Time-to-ready (TTR)

Review by shift & picker. Coach, don't guess.

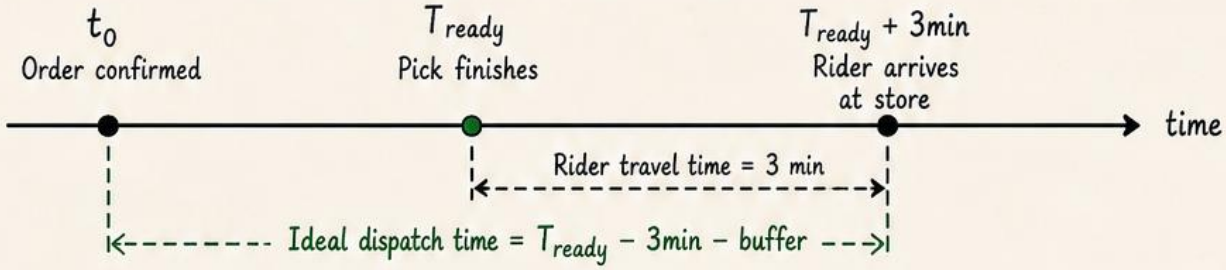
Do not optimize picking like a mega-FC if it breaks the promise.

Sort the aisle, scan the truth, pack to the SLA. ★

12 — RIDER DISPATCH & HANDOFF

Dispatch too early → rider waits at store. Too late → bag waits and the promise dies.

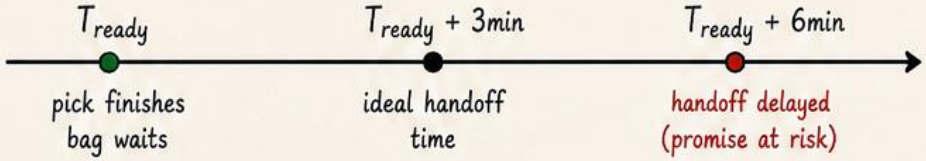
THE TIMING PROBLEM: RIDER vs PICK



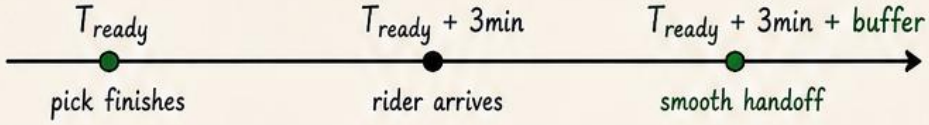
TOO EARLY
rider reaches store before pick is ready



TOO LATE
pick ready, rider not reached

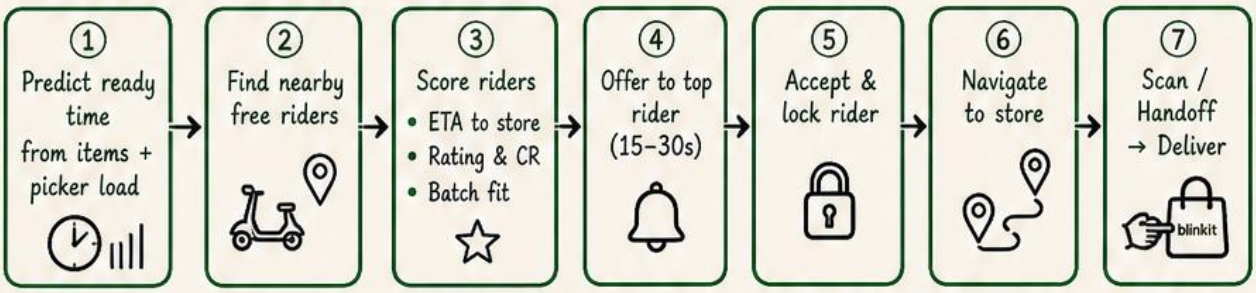


JUST RIGHT
rider arrives on time



Dispatch time = T_{ready} − 3min − buffer (buffer = 30–60s typical)

DISPATCH ASSIGNMENT PIPELINE



CONCURRENCY & SAFETY

- Use Redis lock on rider_id (SETNX + TTL)
- Conditional DB update to move order → DISPATCHED only if rider_id IS NULL
- If lock fails → offer next best rider
- If rider rejects / timeout → release lock and re-offer

BATCHING (USE SPARINGLY)

Batching is rare in ultra-fast.
Only batch if:

- Dropoffs are aligned in direction
- Added delay < ~2–3 min
- Customer promise still protected



★ NOTE: At peak, rider scarcity causes promise inflation — show honest ETA.

Time the scooter to the tote. ★

13 — TRACKING & LIVE ETA

> Customers forgive rain. They do not forgive a silent wrong clock.

1) ETA COMPONENTS (STAGE PIPELINE)

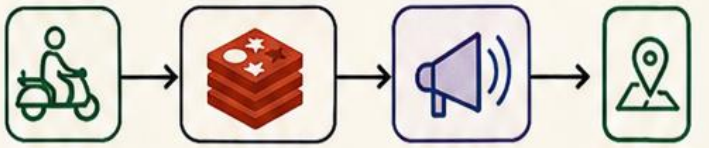
1. **CONFIRMATION**
   ~ 1 min
   Order accepted
2. **KITCHEN QUEUE**
   ~ 5–15 min
   Waiting to be picked
3. **PICKING**
   ~ 6–12 min
   Picker gathering items
4. **PACKING**
   ~ 3 min
   Items packed & sealed
5. **RIDER → STORE**
   ~ 8–15 min
   Rider traveling to store
6. **STORE → CUSTOMER**
   ~ 15–25 min
   On the way to you
7. **LAST-METER**
   ~ 3–7 min
   Near you, finding door

Total ≈ 40–70 min

2) WHAT TRIGGERS AN UPDATE?

Picker starts	Update on: PICKER START
Each major delay	Update on: EACH MAJOR DELAY
Rider assigned	Update on: RIDER ASSIGNED
Rider leaves store	Update on: RIDER LEFT STORE

3) TRACKING PIPELINE



- ✓ Geofence at arrival (store / customer location).
- ✓ OTP / Photo proof (optional) for high-value orders.

4) PRIVACY & VISIBILITY

Show rider location **ONLY AFTER** rider is assigned (handoff policy).

No live location before assignment.

5) QUALITY OF SERVICE (QoS)

- Push on stage changes (via WebSocket).
- Poll fallback every 30–60s if socket is down.
- Client uses last known + ETA to keep UI responsive.

6) CORRECTNESS GUARDRAILS

- ✓ Recompute ETA on each stage event.
- ✓ Monotonic clock per order (never go backward).
- ✓ Cap and floor each component.
- ✓ Show range (low–high) when uncertainty is high.

Recompute on every stage; push the delta.

14 — SUBSTITUTIONS & PARTIAL ORDERS

In grocery, 'out of stock after reserve' still happens — phantom inventory is real.

1. PICKER MARKS MISSING



Picker can't find the item

2. SYSTEM PROPOSES SUBSTITUTES



Same category, size, diet flags, price band

3. CUSTOMER PREFERENCE

AUTO-APPROVE

Use best match

CHAT CONFIRM

Quick confirm

REMOVE ITEM

No substitute

4. ADJUST PAYMENT



5. CONTINUE PICK



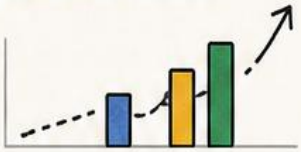
Proceed with remaining items and checkout

WHY PHANTOMS HAPPEN

- Theft or shoplifting
- Misplaced items / wrong location
- Damage or spoilage not updated
- Bad scan / not marked as sold

PREFER CONFIRMING HIGH-VALUE DIFFS

If price difference > threshold (e.g., ₹20 or 15%), always confirm with customer.



NEVER SILENTLY SUBSTITUE

- Baby formula / infant food
- Allergen-containing products
- Prescription / OTC meds
- Religious / fasting-specific items

Always get explicit approval.

REFUNDS ARE LINE-LEVEL

Missing items → remove from order → refund their amount.

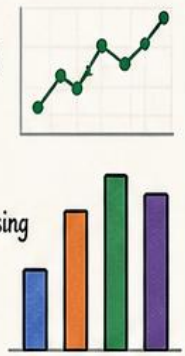
ITEM	STATUS	AMOUNT
Aloo 1kg	Delivered	₹32.00
Amul Milk 1L	Missing	-₹62.00
Bread Brown	Delivered	₹38.00
Refund Total		-₹62.00

TIP: HOLD → CONFIRM → RELEASE

- Keep reservation until missing is confirmed.
- Once confirmed missing, update inventory.
- Release reservation so stock is available again.

METRICS TO WATCH

- Missing rate (by SKU & store)
- Substitute accept rate
- Cancel-after-missing rate



Detect missing early, substitute safely, refund cleanly.

15 — Pricing, Fees & Rain Mode

When demand exceeds pickers or riders, the honest lever is promise time and fees — not fake 10 minutes.

Bill Breakdown (Server-Side Only)

Items total	₹400
Packing fee	₹20
Delivery fee	₹45
Surge / Rain fee	₹60
Taxes (GST)	₹22
Discounts	−₹80
TOTAL	**₹467**

All calculations happen on server. Client shows final numbers only.

Surge / Rain — Triggers

- ✓ Picker queue length is high
- ✓ Rider shortage in the zone
- ✓ Heavy rain / bad weather
- ✓ Zone heat map is red (orders > capacity)

Surge / Rain — Effects

- Raise delivery fee and/or inflate promise time
- Optionally soft-close store if capacity is overwhelmed.

Rain Mode



- ✓ Slower travel assumptions (rides + traffic)
- ✓ More buffer in promise times
- ✓ Safety first — fewer risky routes
- ✓ Show clear messaging to user

Goal: protect safety and keep promises honest.

Promos & Coupons

- Apply atomically on server. No stacking hacks.
- Coupon budget is inventory.
- Membership (e.g., Blinkit Plus):
  - Free delivery if order ≥ ₹99
  - Else delivery fee applies
- Never show crossed-out fake prices. Keep it real.



NEVER SHOW 10 MIN IF MODEL PREDICTS 25.

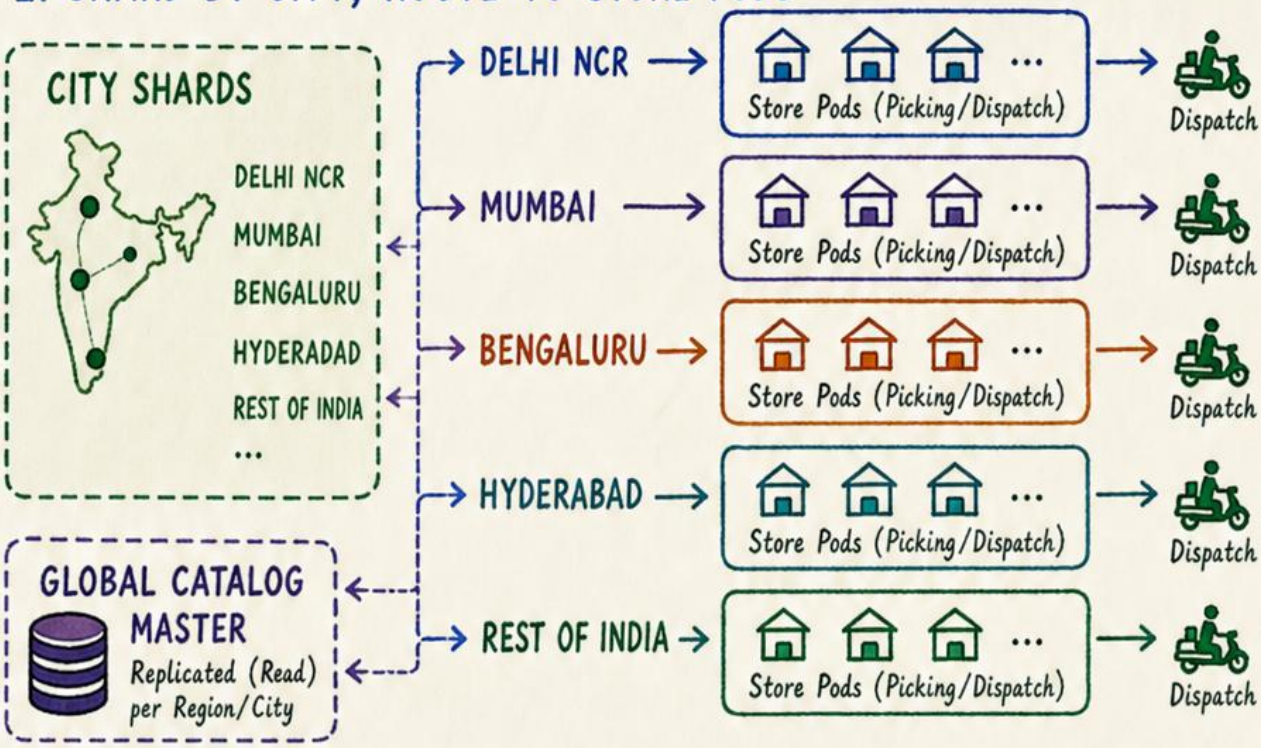
Better to be honest and late than lie and disappoint.

★ Price the scarcity; keep the clock honest. ★

16 — SCALING & CACHING

Shard by city/store. Hot milk SKUs will still try to melt your locks.

1. SHARD BY CITY, ROUTE TO STORE PODS



2. CACHE LAYERS (HOT PATH)

CDN	Images, banners, static assets
Redis	Store category pages, deals, landing (short TTL)
Inventory Flags	In-stock / low / OOS (very short TTL, read-through)
Carts	User cart, offers, coupons (short TTL)
Rider Geo	Rider location buckets, ETA grids (short TTL)

3. HOT PATH (SYNC)

1. Reserve Inventory (tiny txn)
2. Payment (authorize/capture)
3. Pick Scan (item-level scan)
4. Assign Rider (nearest available)

4. ASYNC (EVENTS)

- Notifications (email / push / SMS)
- Analytics (events to lake)
- Recommendations (model updates, feeds)
- Event Log (Kafka topics)

5. HOT SKU STRATEGIES

- ✓ Row-level conditional updates (UPDATE ... WHERE stock > 0)
- ✓ Avoid cross-store transactions (store is the unit of truth)
- ✓ Inventory Service (optional) colocated per city shard (low-latency, tiny transactions)

6. FESTIVAL / IPL PREP

- ✓ Pre-scale store pods & workers
- ✓ Pre-stock hot SKUs (milk, bread, eggs, chips, cola)
- ✓ Raise inventory buffers (cover spikes)
- ✓ Warm caches & CDN
- ✓ Chaos drills & runbooks

7. CACHE RULES

- Short TTL for freshness
- Read-through for inventory flags
- Write-through for critical updates
- Inval by key pattern (store:sku:*)
- Dogpile protection (locks + jittered refresh)

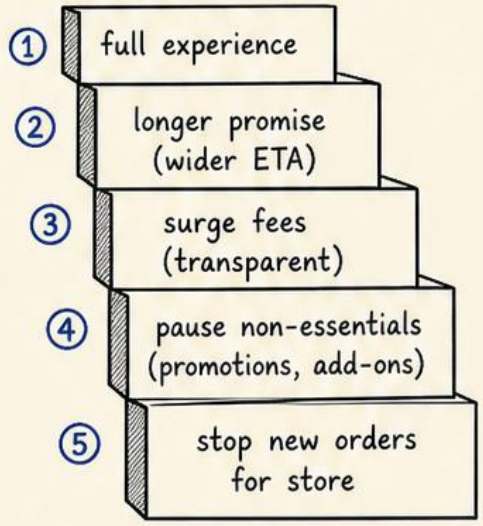
☆ *City shards, store throughput, tiny inventory transactions.* ☆

17 — FAILURES & RELIABILITY

Fail on promise honesty before you fail on silent cancellation.

WHAT BREAKS	→ WHAT WE DO
Inventory service down	stop checkout (fail closed) keep browse cached
Payment timeout	enquire; release reserve on fail
Picker app offline	reassign task; alert store OM
Rider no-show	reoffer; inflate ETA
Maps ETA bad	use fallback speeds + buffer
Kafka lag	delay analytics not picking
Store power/network outage	soft-close store polygon
Partial stock fail at reserve	return line errors

GRACEFUL DEGRADATION LADDER



Note: Move down the ladder to protect promises. Move up only when stable.

PROTECT THE HOT PATH

- Timeouts everywhere
- Circuit breakers on every downstream call
- Idempotency for all state-changing actions (charge, reserve, notify)
- Bulkheads to search, maps, payments
- Fallbacks with safe defaults
- Rate limits per user and per partner

WHAT TO MONITOR

- Error rate by service
- P95 / P99 latency on hot path
- Timeout rate
- Cancellation rate by reason
- Payment success rate
- Rider supply-demand ratio per zone
- Promised vs actual ETA delta

OPERATIONS MATTER

A live control room can pause a store, reassign riders, extend an ETA or issue a refund. Build the tools. Practice the drills. Trust the runbook.

★ Fail closed on stock. Fail honest on ETA. ★

18 — METRICS, OPS & STORE CONTROL

Quick commerce is operated minute-by-minute at the store, not only at the region.

NORTH-STAR METRICS

PROMISE KEPT RATE	% orders delivered within promise
MEDIAN / P90 DELIVERY TIME	minutes from order to drop
CANCEL RATE	% orders cancelled
ITEM FILL RATE	items delivered vs requested
OVERSELL INCIDENTS (= 0)	must be zero every day
PICKER LINES / HOUR	completed picker lines per hour
RIDER UTILIZATION	active time / online time

STORE CONSOLE (REAL-TIME CONTROL)

STORE	LIVE ORDERS	PROMISE BUFFER
HSR Layout OPEN	46 (last 15 min)	+5 min

- **OPEN / CLOSE STORE** — control store availability
- **PAUSE CATEGORIES** — pause low-stock / high-risk categories
- **ADJUST PROMISE BUFFER** — + / − minutes based on load
- **STAFFING TOGGLES** — call pickers / packers on standby

INVENTORY OPERATIONS

- **CYCLE COUNT** — accuracy check by aisle / bin
- **WASTE MANAGEMENT** — mark & track waste reasons
- **GRN & RESTOCK** — good receipt note & auto restock
- **SHRINKAGE TRACKING** — track loss, damage, expiry

ALERTING & ESCALATIONS

- **PENDING PICK SPIKE** — pending orders > threshold for 10 min
- **RIDER DEFICIT** — rider ETA > promise buffer
- **PAYMENT SUCCESS DROP** — payment success rate < baseline
- **RESERVE FAILURE SPIKE** — inventory reserve failure > threshold

SAFETY & COMPLIANCE

- **OTP DELIVERY** — on-time delivery discipline every day
- **BANNED ITEMS RULES** — block sale & flag violations (tobacco, gutkha, etc.)
- **TEMPERATURE BAGS** — use temperature bags for cold chain items

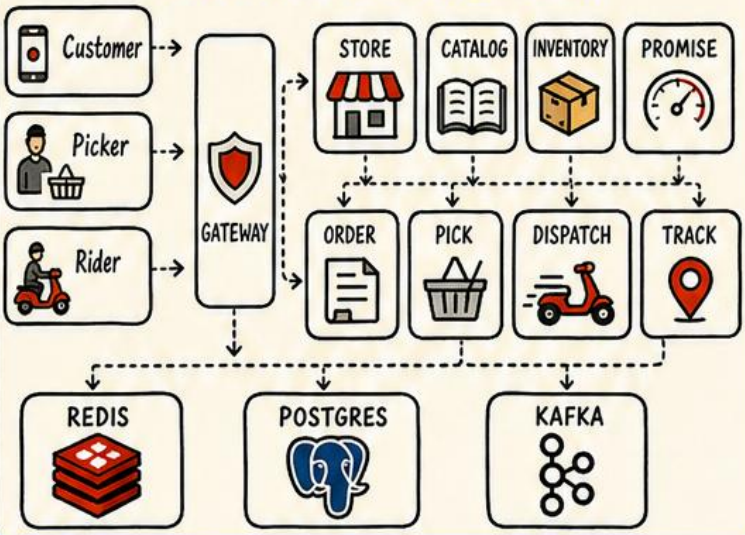
★ Measure fill rate and promise-kept — daily. ★

19 — RECAP & INTERVIEW CHEAT SHEET

THE WHOLE SYSTEM IN ONE LINE

Pick a nearby dark store, reserve real stock, pick in aisle order, time the rider to the bag, keep the 10-minute clock honest.

MINI ARCHITECTURE RECAP



TOP 10 TALKING POINTS

1. Dark store first — distance is destiny.
2. Store-local inventory — no global fiction.
3. Atomic reserve with TTL — prevent oversell.
4. Promise = pick time + ride time + buffer.
5. Picker path = aisle order, slotting & batching.
6. Timed dispatch — rider at door, not in parking.
7. Batching only when idle and ultra-close.
8. Smart substitutions — customer choice first.
9. Rain, surge, events — protect the promise.
10. City/store sharding — scale without chaos.

COMPARE AT A GLANCE

vs ZOMATO (FOOD DELIVERY)	vs AMAZON (ECOMMERCE)
Kitchen prepares after order	Warehouses, ship from far
Many restaurants, no inventory control	Large catalog, not store-local
Menu changes frequently	Static catalog, long tail
ETA depends on restaurant prep	Days of delivery, not minutes
Rider pickup from restaurant	Long distance, multi-hop

CHEAT SHEET: NUMBERS TO REMEMBER

- 2.5M+ orders/day at peak
- ~10K orders/sec at peak
- 40K+ location writes/sec
- 5–7 km delivery radius
- 30 s rider offer window
- 2 min restaurant acceptance window
- 10-minute promise (P95)

MISTAKES TO AVOID

- ✗ Global catalog without store context
- ✗ Fake 10 minutes (no real buffer)
- ✗ Forgetting the picker experience
- ✗ Long-running inventory transactions
- ✗ Silent substitutions without choice

QUICK Q & A

Q: How do we prevent oversell?
A: Atomic reserve with TTL at store+sku level.

Q: Why not reserve when user adds to cart?
A: Carts expire. Reserve only on order placement.

Q: When do we assign a rider?
A: After pick starts or is about to finish. Rider reaches store just-in-time.

Q: How do we handle phantom inventory?
A: Store-local truth. Reconcile and compensate fast.

Q: What if a store runs low or closes early?
A: Soft-close: stop new promises, fulfill existing, reroute to nearest store if possible.

